

Event Booking System API Documentation

Project Summary

This project implements a RESTful API for an Event Booking System using Laravel. It features **Role-Based Access Control (RBAC)** for Admin, Organizer, and Customer roles, manages events and tickets, handles bookings with concurrency checks, processes mock payments, and includes performance optimizations via **Queues** (for notifications) and **Caching** (for event listings).

1. Authentication & Users

Authentication is handled via **Laravel Sanctum**. Protected endpoints require a Bearer Token obtained after successful login.

Endpoint	Method	Access	Description
/api/register	POST	Public	Create a new user (default role: customer).
/api/login	POST	Public	Authenticate and receive a token (Bearer).
/api/logout	POST	Authenticated	Revoke the current access token.
/api/me	GET	Authenticated	Retrieve authenticated user details.

2. Role-Based Access Control (RBAC)

Access is controlled via the `RoleMiddleware` applied to routes.

Role	Access Level	Permissions
Admin	Highest	Full CRUD access across all models.
Organizer	Medium	Can CRUD their own Events and associated Tickets.
Customer	Limited	Can view all Events/Tickets, create Bookings, cancel bookings, and process payments.

3. Event and Ticket APIs

The event listing endpoint is cached for 5 minutes and supports filtering.

Endpoint	Method	Access	Description
/api/events	GET	Public	List all events. Supports optional query params: ? search=keyword (via searchByTitle scope) and ? date=YYYY-MM-DD (via filterByDate scope).
/api/events/{id}	GET	Public	View a specific event with its tickets.
/api/events	POST	Admin, Organizer	Create a new event. Triggers cache invalidation.
/api/events/{id}	PUT	Admin, Organizer (Owner)	Update an event. Triggers cache invalidation.
/api/events/{id}	DELETE	Admin, Organizer (Owner)	Delete an event. Triggers cache invalidation.
/api/events/{event_id}/tickets	POST	Admin, Organizer (Owner)	Create a new ticket for an event.
/api/tickets/{id}	PUT	Admin, Organizer (Owner)	Update a ticket.
/api/tickets/{id}	DELETE	Admin, Organizer (Owner)	Delete a ticket. Fails if active bookings exist.

4. Booking and Payment APIs

Endpoint	Method	Access	Description
/api/tickets/{ticket_id}/bookings	POST	Customer	Create a booking. Middleware (PreventDoubleBooking) ensures the user cannot create a second pending/confirmed booking for the same ticket. Concurrency check prevents overbooking.
/api/bookings	GET	Customer	View the authenticated customer's list of bookings.
/api/bookings/{id}/cancel	PUT	Customer	Cancel a booking. Frees up ticket quantity.

/api/ bookings/ {id}/payment	P OS T	Authe nticat ed	Process mock payment using the PaymentService . Updates booking status to 'confirmed' on success and dispatches an email notification to the queue .
/api/ payments/{id}	G ET	Authe nticat ed	View payment details (Access restricted by ownership/event control).

5. System Features & Optimization

A. Middleware, Services & Traits (Section 4)

- **PreventDoubleBooking Middleware:** Attached to the booking creation route to ensure a user only holds one active booking (pending or confirmed) per ticket type.
- **PaymentService :** A service layer abstraction for the mock payment gateway with an 80% success simulation rate.
- **CommonQueryScopes Trait:** Used by the `Event` model to easily apply `searchByTitle()` and `filterByDate()` logic in controllers.

B. Notifications, Queues & Caching (Section 5)

- **Queued Notifications:** The `BookingConfirmed` notification implements `ShouldQueue` . When a payment succeeds, the email is placed on the database queue, ensuring the API response is instant.
- **Event Caching:** The `GET /api/events` endpoint uses `Cache::remember()` to store results for 5 minutes.
- **Cache Invalidation:** All event and ticket modification endpoints (`POST/PUT/DELETE`) implement `Cache::flush()` to ensure cached listings are always up-to-date.

6. Submission Checklist

1. [✓] All models, migrations, and factories are complete.
2. [✓] All APIs implemented with correct routing and RBAC.
3. [✓] Middleware, Services, and Traits are integrated.
4. [✓] Notifications, Queues, and Caching mechanisms are set up.